

简易编译器 QCompiler 实现及验证实验报告

任务类型：A，递归下降语法分析 + QAST + QTAC + ARM64
实现语言：Python 3
实验目录：`labs/lab2`

一、实验目的

本实验围绕自定义 QL 语言实现一个简易编译器 QCompiler，目标是打通从源程序到目标汇编的主要编译流程。实验内容覆盖词法分析、语法分析、抽象语法树构造、中间代码生成、符号表管理、ARM64 目标代码生成以及简单代码优化。

通过本实验，重点完成以下目标：

1. 理解 QL 源语言的词法、文法和语义约束。
2. 实现 QL 到 Token 序列的词法分析。
3. 采用任务 A 路线，实现递归下降语法分析并构造 QAST。
4. 遍历 QAST 生成 QTAC 三地址中间代码和符号表。
5. 利用 Lab1 中已经完成的 QTAC 到 ARM64 指令模板思路，实现目标代码生成。
6. 对生成的 ARM64 汇编做简单语义保持优化。
7. 使用 qsort 测试程序验证完整编译链路。

二、实验环境

本次实现和本地验证环境如下：

项目	内容
操作系统	Linux
Python 版本	Python 3.14
开发工具	Codex、Shell、VSCode
实现语言	Python
目标架构	ARM64 AArch64
测试框架	<code>unittest</code>
实机测试	OrangePi AIPro(8T)

三、实验路线选择

实验指导书给出了 A、B、C 三种任务路线。本实现选择 A 路线：

QL 源程序

- Token 序列
- 递归下降语法分析
- QAST
- QTAC + 符号表
- ARM64 汇编
- 优化后 ARM64 汇编

选择 A 路线的原因如下：

- 1. Lab1 已经实现了 QTAC 到 ARM64 的后端雏形，A 路线可以自然复用该部分工作。
- 2. 递归下降分析器便于手工处理 qsort 测试程序中的函数、数组、控制流和表达式。
- 3. 生成 QAST 后，可以清晰分离前端语法分析和中间代码生成，便于调试和报告展示。

四、工程文件结构

本次实验的主要文件如下：

文件或目录	作用
qcompiler/lexer.py	QL 词法分析器
qcompiler/parser.py	递归下降语法分析器
qcompiler/ast_nodes.py	QAST 节点定义和 JSON 序列化
qcompiler/qtac.py	QAST 到 QTAC 的中间代码生成
qcompiler/arm.py	QTAC 到 ARM64 汇编生成和简单优化
qcompiler/compiler.py	编译器完整流水线接口
qcompiler/cli.py	命令行入口
samples/qsrt.q1	qsrt 测试源程序
outputs/qsrt/	qsrt 各阶段输出文件
tests/test_qcompiler.py	自动化回归测试
docs/report.md	本实验报告

运行完整编译流程的命令为：

```
python -m qcompiler samples/qsrt.q1 -o outputs/qsrt
```

五、词法分析器实现

词法分析器位于 `qcompiler/lexer.py`。其输入为 QL 源程序文本，输出为 Token 序列。

5.1 Token 类型

本实现识别以下 Token：

Token 类型	示例
关键字	<code>int</code> 、 <code>void</code> 、 <code>if</code> 、 <code>else</code> 、 <code>while</code> 、 <code>return</code>
标识符	<code>qsort</code> 、 <code>partition</code> 、 <code>arr</code> 、 <code>pivot</code>
整数常量	<code>0</code> 、 <code>1</code> 、 <code>9</code> 、 <code>10</code>
算术运算符	<code>+</code> 、 <code>-</code> 、 <code>*</code> 、 <code>/</code>
赋值运算符	<code>=</code> 、 <code>+=</code>
关系运算符	<code><</code> 、 <code><=</code> 、 <code>></code> 、 <code>>=</code> 、 <code>==</code> 、 <code>!=</code>
逻辑运算符	<code>!</code> 、 <code>&&</code> 、 <code> </code>
界符	<code>{</code> 、 <code>}</code> 、 <code>(</code> 、 <code>)</code> 、 <code>[</code> 、 <code>]</code> 、 <code>;</code> 、 <code>,</code>

词法分析器还处理了两类工程细节：

- 1. 跳过 `//` 行注释。
- 2. 将实验材料中可能出现的 Unicode 减号 `−`、`‑` 统一归一化为 ASCII `-`，将 `×` 归一化为 `*`。

5.2 Token 输出格式

Token 输出文件为：

```
outputs/qsort/qsort.tokens.txt
```

输出格式为：

```
TokenType  TokenText  line:column
```

qsort 程序共生成 235 行 Token 输出。

六、语法分析与 QAST 生成

语法分析器位于 `qcompiler/parser.py`，采用递归下降方式实现。语法分析器接受词法分析器产生的 Token 流，输出由 `qcompiler/ast_nodes.py` 中 dataclass 表示的 QAST。

6.1 支持的 QL 语法结构

本实现覆盖 qsort 测试程序和实验文法中的核心结构：

1. 全局变量和全局数组声明，例如 `int a[10];`
2. 函数定义，例如 `void qsort(int arr[]; int low; int high;)`
3. 函数形参，包括普通变量和数组形参。
4. 局部变量声明。
5. 空语句、表达式语句、复合语句。
6. `if`、`if else`、`while`、`return`。
7. 函数调用和尾随逗号形式，例如 `qsort(a[], 0, 9,);`
8. 数组访问和数组退化，例如 `arr[i]` 和 `arr[]`。
9. 赋值、算术、关系、逻辑和一元运算。

6.2 递归下降文法改写与 LL(1) 化说明

实验给出的 QL 文法中，表达式、条件表达式、声明列表和语句列表更接近自然描述形式，直接实现递归下降时会遇到两个问题：

1. 表达式中存在类似 `E -> E + T`、`C -> C && C` 的左递归形式。
2. 函数定义、变量声明、函数调用、数组访问等结构在前几个 Token 上有公共前缀，需要提取左公因子后再判断分支。

因此，本实现没有直接照抄原始文法，而是将 qsort 所需的 QL 子集改写为适合递归下降分析的优先级分层文法。条件表达式和算术表达式的核心改写如下：

```
C      → C_or
C_or   → C_and C_or'
C_or'  → || C_and C_or' | ε
C_and  → C_not C_and'
C_and' → && C_not C_and' | ε
C_not  → ! C_not | C_rel
C_rel  → E (rop E)?

E      → Assign
Assign → Add ((= | +=) Assign)?
Add    → Mul Add'
Add'   → (+ | -) Mul Add' | ε
Mul    → Unary Mul'
Mul'   → (* | /) Unary Mul' | ε
Unary   → - Unary | Primary
Primary → number | id PrimaryTail | ( C )
```

全局声明和函数定义都以类型和标识符开头，因此需要提取左公因子：

```
TopItem  → T id TopTail
TopTail  → ( ParamList ) Block | ArraySuffix ;
ArraySuffix → [ number? ] | ε
```

读到 `T id` 后，若下一个 Token 是 `(`，则进入函数定义；若下一个 Token 是 `[` 或 `;`，则进入变量或数组声明。这个分支判断对应 `parser.py` 中的 `parse()` 和 `parse_function_after_header()`。

语句入口也按首 Token 区分：

非终结符	产生式	SELECT / FIRST 入口	对应实现
S	;	;	parse_statement()
S	Block	{	parse_block()
S	if (C) S else S	if	parse_statement()
S	while (C) S	while	parse_statement()
S	return E ;	return	parse_statement()
S	E ;	id、number、(、-、!	parse_statement()
TopTail	(ParamList) Block	(	parse_function_after_header()
TopTail	ArraySuffix ;	[、;	parse()
PrimaryTail	(ArgList)	(	parse_primary()
PrimaryTail	[Index] 或 []	[	parse_primary()
PrimaryTail	ε	运算符、)、;、,、]	parse_primary()

从上表可以看到，改写后的 qsort 核心子集在各个选择点上 SELECT 集互不相交。表达式部分通过优先级分层消除了左递归，函数/变量声明和标识符后缀通过左公因子提取后只需要观察 1 个后继 Token 即可确定分支，因此满足递归下降分析的 LL(1) 入口要求。

本实验没有额外生成独立的 SELECT 集输出文件；SELECT 集和文法改写结果在报告中列出，实际递归下降分析器源码位于 `qcompiler/parser.py`，QAST 输出文件为 `outputs/qsort/qsort.ast.json`。

6.3 QAST 节点设计

QAST 节点使用 Python dataclass 定义，主要包括：

节点	含义
Program	整个 QL 程序
VarDecl	变量或数组声明
FunctionDecl	函数定义
Param	函数形参
Block	复合语句块
IfStmt	条件语句
WhileStmt	循环语句
ReturnStmt	返回语句
ExprStmt	表达式语句
Number	整数常量
Var	变量引用
ArrayAccess	数组访问
ArrayDecay	数组退化为地址
Call	函数调用
UnaryOp	一元运算
BinaryOp	二元运算
Assign	赋值表达式

QAST 输出为 JSON 文件：

```
outputs/qsrt/qsrt.ast.json
```

该文件保留了每个节点的 `node` 字段，便于观察 AST 层次。例如 qsrt 程序中包含 `FunctionDecl`、`WhileStmt`、`IfStmt`、`Call`、`ArrayAccess` 等节点。

七、QTAC 中间代码生成

QTAC 生成器位于 `qcompiler/qtac.py`。其输入为 QAST，输出为 QTAC 程序和符号表。

7.1 QTAC 指令子集

本实现生成的 QTAC 指令包括：

```
LABEL l;  
GOTO l;  
IF q rop a THEN l1 ELSE l2;  
PAR a;  
q = CALL d, k;  
RETURN a;  
q = a;  
q = q aop a;  
q = M[q];  
M[q] = a;
```

这些指令覆盖 `qsort` 程序所需的函数调用、条件跳转、循环跳转、算术计算、数组访存和返回。

7.2 数组访问翻译

数组元素访问 `arr[i]` 被翻译为：

```
t0 = i * 8;  
t1 = arr + t0;  
t2 = M[t1];
```

其中乘以 8 是因为 ARM64 后端使用 64 位寄存器和 `LDR`、`STR` 指令，每个整数按 8 字节处理。

数组元素赋值 `arr[i] = v;` 被翻译为：

```
t0 = i * 8;  
t1 = arr + t0;  
M[t1] = v;
```

7.3 qsort QTAC 输出摘要

QTAC 输出文件为：

```
outputs/qsort/qsort.qtac
```

部分输出如下：

```
i80 a;

define partition(arr, low, high){
LABEL partition;
t0 = high * 8;
t1 = arr + t0;
t2 = M[t1];
pivot = t2;
t3 = low - 1;
i = t3;
t4 = low - 1;
j = t4;
LABEL l0;
t5 = j + 1;
j = t5;
IF j < high THEN l1 ELSE l2;
...
RETURN t13;
}
```

qsort 程序最终生成 89 行 QTAC。

八、符号表生成

符号表输出文件为：

```
outputs/qsort/qsort.symbols.txt
```

qsort 程序的符号表摘要如下：

```
globals:
  a: 80 bytes
functions:
  swap:
    params: arr, i, j
    locals: temp
  partition:
    params: arr, low, high
    locals: pivot, i, j
  qsort:
    params: arr, low, high
    locals: p
```

其中全局数组 `a[10]` 按 10 个 8 字节整数分配，共 80 字节。

九、ARM64 目标代码生成

ARM64 后端位于 `qcompiler/arm.py`。该部分在 Lab1 后端的基础上扩展为可处理多函数、全局数组和 `_start` 程序入口。

9.1 Lab1 工作复用

Lab1 中已经实现了以下思路：

1. 对 QTAC 语句进行模板分类。
2. 根据模板生成对应的 ARM64 指令。
3. 使用 `x0` 到 `x7` 作为参数寄存器。
4. 使用 `x0` 作为返回值寄存器。
5. 为变量分配寄存器或栈槽。

Lab2 后端复用了这些思想，并做了以下扩展：

1. 支持多个函数分别生成序言和尾声。
2. 支持 `.section .data` 中的全局数组。
3. 支持 `_start` 作为顶层语句入口。
4. 支持函数内局部标签重命名，例如 `l0` 生成 `.L_partition_l0`，避免不同函数之间标签冲突。
5. 支持数组地址作为全局符号地址或参数地址。

9.2 QTAC 模板识别与 σ -DFA 对应关系

实验输出要求中提到的“ σ -DFA 转换 QTAC 为 ARM 代码”，本实现采用的是结构化模板识别方式完成同一类工作。Lab1 中对文本 QTAC 语句进行模板分类；Lab2 中，`qcompiler/qtac.py` 在生成 QTAC 时同步生成 `TACInstruction(template, data)`，其中 `template` 字段已经记录该条 QTAC 属于哪一种指令模板。ARM 后端 `qcompiler/arm.py` 直接根据模板分发到对应的 ARM64 生成逻辑。

这种方式避免了在后端再次用字符串重新解析 QTAC，但语义上仍然对应“先识别 QTAC 指令类型，再按类型生成 ARM 指令”的 σ -DFA 模板识别流程。核心模板对应关系如下：

QTAC 形式	模板类型	ARM64 生成结果
<code>LABEL 1;</code>	<code>T_LABEL</code>	输出函数标签或局部跳转标签
<code>GOTO 1;</code>	<code>T_GOTO</code>	<code>B label</code>
<code>IF q rop a THEN l1 ELSE l2;</code>	<code>T_IF</code>	<code>CMP</code> + 条件分支 + 无条件分支
<code>PAR a;</code>	<code>T_PAR</code>	暂存调用实参
<code>q = CALL d, k;</code>	<code>T_CALL</code>	将实参放入 <code>x0</code> 到 <code>x7</code> ，执行 <code>BL d</code> ，再保存 <code>x0</code>
<code>RETURN a;</code>	<code>T_RETURN</code>	将返回值放入 <code>x0</code> ，跳到函数尾声
<code>q = a;</code>	<code>T_MOVE</code>	常量加载或寄存器移动
<code>q = q aop a;</code>	<code>T_AOP</code>	<code>ADD</code> 、 <code>SUB</code> 、 <code>MUL</code> 、 <code>SDIV</code> 等算术指令
<code>q = M[q];</code>	<code>T_LDR0</code>	<code>LDR</code>
<code>M[q] = a;</code>	<code>T_STR0</code>	<code>STR</code>

因此，附件中的 QTAC 文件可以直接人工查看文本形式；程序内部则使用结构化模板字段完成等价的模板识别和 ARM 生成。

9.3 函数调用约定

本实现遵循简化的 AArch64 调用约定：

项目	实现方式
参数传递	<code>x0</code> 到 <code>x7</code>
返回值	<code>x0</code>
栈帧	使用 <code>x29</code> 作为帧指针， <code>x30</code> 保存返回地址
局部值	优先分配到 <code>x19</code> 到 <code>x28</code> ，不足时使用栈槽
程序退出	<code>_start</code> 中使用 Linux syscall 93

函数序言示例：

```
qsort:
    STP X29, X30, [SP, #-16]!
    MOV X29, SP
    SUB SP, SP, #80
    STR X19, [SP, #0]
    ...
```

函数尾声示例：

```
.L_return_qsort:
    LDR X19, [SP, #0]
    ...
    ADD SP, SP, #80
    LDP X29, X30, [SP], #16
    RET
```

9.4 ARM64 输出

未优化 ARM64 汇编输出文件为：

```
outputs/qsort/qsort.s
```

优化后 ARM64 汇编输出文件为：

```
outputs/qsort/qsort_opt.s
```

本次 qsort 程序生成：

文件	行数
qsort.s	274
qsort_opt.s	229

十、汇编优化

本实现包含一个 ARM64 peephole 优化 pass，目标是在不改变语义的前提下降低冗余指令数量，并展示比单纯删除空跳转更明显的优化效果。

当前实现了以下优化：

- 1. 删除自拷贝指令，例如：

```
MOV Xn, Xn
```

2. 删除直接跳到下一条标签的无条件分支，例如：

```
        B .L_x
.L_x:
```

3. 强度削弱：将乘以 8 的地址偏移计算改为左移 3 位。例如：

```
MOV X10, #8
MUL X11, X9, X10
```

优化为：

```
LSL X11, X9, #3
```

4. 相邻指令合并：在安全范围内合并 scratch 寄存器上的临时拷贝。例如将“先写入临时寄存器、再立即搬运”的模式合并为直接写入目标寄存器。

优化前后行数变化如下：

文件	行数
未优化 ARM	274
优化后 ARM	229

优化后减少 45 行汇编，约减少 16.4%。优化后的 `qsort_opt.s` 中可以看到多条 `LSL` 指令，说明数组下标乘以 8 的地址计算已经被替换为更直接的移位指令。优化规则限定在相邻指令和 scratch 寄存器范围内，避免跨函数调用或跨基本块进行不安全替换。

十一、测试与验证

11.1 自动化测试

测试文件为：

```
tests/test_qcompiler.py
```

测试内容包括：

1. 词法分析器能够跳过注释并归一化 Unicode 减号。
2. 数组赋值能够生成 `M[...]` 形式的 QTAC 内存写指令。
3. qsort 能够完整生成 Token、AST、QTAC、ARM 和优化 ARM。
4. ARM64 函数入口标签不会重复生成。
5. ARM64 函数内部控制流标签会加函数名前缀，避免跨函数冲突。
6. 优化器能够执行乘以 8 到左移 3 位的强度削弱，并显著减少汇编行数。
7. 优化器不会删除函数参数从 `x0`、`x1`、`x2` 到局部保存寄存器的初始化指令。

测试命令：

```
python3 -m unittest discover -s tests -p "test_*.py"
```

测试结果：

```
.....
Ran 7 tests in 0.027s

OK
```

11.2 Python 编译检查

检查命令：

```
python3 -m compileall qcompiler tests
```

检查结果通过，输出为：

```
Listing 'qcompiler'...
Listing 'tests'...
```

11.3 qsort 各阶段输出统计

生成命令：

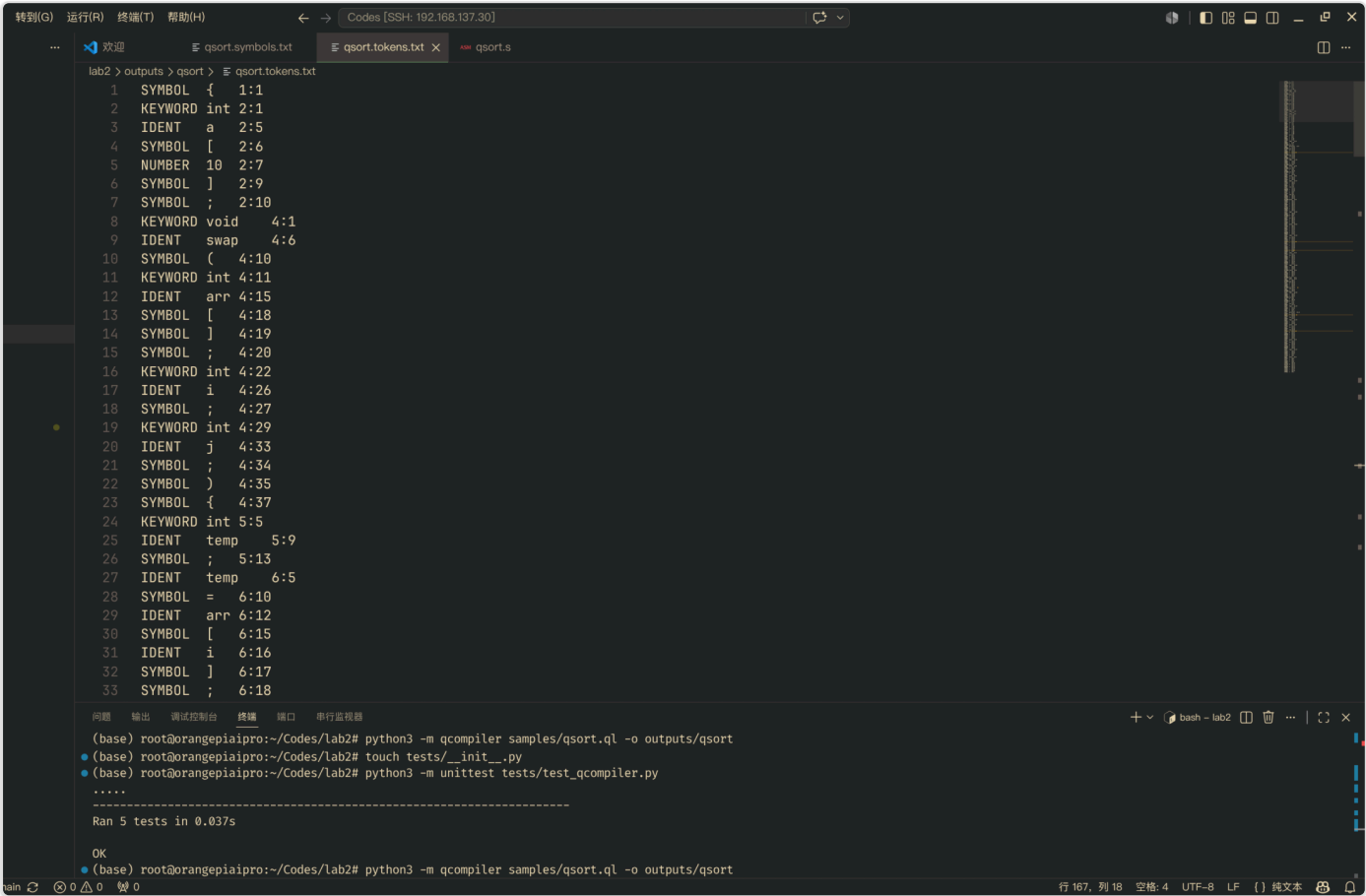
```
python3 -m qcompiler samples/qsrt.ql -o outputs/qsrt
```

输出文件统计：

```
235 outputs/qsrt/qsrt.tokens.txt
568 outputs/qsrt/qsrt.ast.json
 89 outputs/qsrt/qsrt.qtac
274 outputs/qsrt/qsrt.s
229 outputs/qsrt/qsrt_opt.s
1395 total
```

11.4 关键中间结果内容展示

qsrt.tokens.txt



```

define swap(arr, i, j){
  LABEL swap;
  t0 = i * 8;
  t1 = arr + t0;
  t2 = M[t1];
  temp = t2;
  t3 = j * 8;
  t4 = arr + t3;
  t5 = M[t4];
  t6 = i * 8;
  t7 = arr + t6;
  M[t7] = t5;
  t8 = j * 8;
  t9 = arr + t8;
  M[t9] = temp;
  RETURN 0;
}

define partition(arr, low, high){
  LABEL partition;
  t0 = high * 8;
  t1 = arr + t0;
  t2 = M[t1];
  pivot = t2;
  t3 = low - 1;
  i = t3;
  t4 = low - 1;
  j = t4;
  LABEL l0;
  t5 = j + 1;
  j = t5;

```

```

1  define qsort(arr, low, high){
2  LABEL qsort;
3  IF low < high THEN l0 ELSE l1;
4  LABEL l0;
5  PAR arr;
6  PAR low;
7  PAR high;
8  t0 = CALL partition, 3;
9  p = t0;
10 PAR arr;
11 PAR low;
12 t1 = p - 1;
13 PAR t1;
14 t2 = CALL qsort, 3;
15 PAR arr;
16 t3 = p + 1;
17 PAR t3;
18 PAR high;
19 t4 = CALL qsort, 3;
20 GOTO l2;
21 LABEL l1;
22 LABEL l2;
23 RETURN 0;
24 }

```

```

PAR a;
PAR 0;
PAR 9;
t0 = CALL qsort, 3;

```

qsort.s & qsort_opt.s

```

102     MOV X19, X0
103     MOV X20, X1
104     MOV X21, X2
105     MOV X9, X21
106     ✦ MOV X10, #8
107     MUL X11, X9, X10
108     MOV X22, X11
109     MOV X9, X19
110     MOV X10, X22

```

```

104     B.LT .L_partition_l1
105     B .L_partition_l2
106 .L_partition_l1:
107     LDR X9, [SP, #80]
108     ✦ LSL X11, X9, #3
109     STR X11, [SP, #96]
110     MOV X9, X19
111     LDR X10, [SP, #96]
112     ADD X11, X9, X10
113     STR X11, [SP, #104]

```

上面两张图中，上面是原版，下面是优化过后的，可以看到两行的 **MOV** **MUL** 被优化成了一行 **LSL** 指令。

```

lab2 > outputs > qsort > asm qsort_opt.s

1  .section .data
2  .align 3
3  a: .skip 80
4
5  .section .text
6  .global swap
7  swap:
8      STP X29, X30, [SP, #-16]!
9      MOV X29, SP
10     SUB SP, SP, #112
11     STR X19, [SP, #0]
12     STR X20, [SP, #8]
13     STR X21, [SP, #16]
14     STR X22, [SP, #24]
15     STR X23, [SP, #32]
16     STR X24, [SP, #40]
17     STR X25, [SP, #48]
18     STR X26, [SP, #56]
19     STR X27, [SP, #64]
20     STR X28, [SP, #72]
21     MOV X19, X0
22     MOV X20, X1
23     MOV X21, X2
24     LSL X11, X20, #3
25     MOV X22, X11
26     MOV X9, X19
27     MOV X10, X22
28     ADD X9, X9, X10
29     LDR X25, [X9]
30     LSL X11, X21, #3

```

除此之外，以下这些片段（**.global _start** **_start** **.align** 等）证明生成的代码有一个完整的结构。

11.5 ARM 实机环境截图

在 ARM64 设备上进入实验目录后，先记录系统架构和工具链版本：

```
pwd
uname -m
python3 --version
as --version | head -n 1
ld --version | head -n 1
```

```
(base) root@orangepiaipro:~/Codes/lab2# ls
docs  outputs  qcompiler  samples  tests
(base) root@orangepiaipro:~/Codes/lab2# pwd
/root/Codes/lab2
(base) root@orangepiaipro:~/Codes/lab2# python3 -m unittest discover -s tests -p "test_*.py"
(base) root@orangepiaipro:~/Codes/lab2# uname -m
aarch64
(base) root@orangepiaipro:~/Codes/lab2# python3 --version
Python 3.9.2
(base) root@orangepiaipro:~/Codes/lab2# as --version
GNU assembler (GNU Binutils for Ubuntu) 2.38
Copyright (C) 2022 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or later.
This program has absolutely no warranty.
This assembler was configured for a target of `aarch64-linux-gnu'.
(base) root@orangepiaipro:~/Codes/lab2# ld --version
GNU ld (GNU Binutils for Ubuntu) 2.38
Copyright (C) 2022 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) a later version.
This program has absolutely no warranty.
(base) root@orangepiaipro:~/Codes/lab2#
```

11.6 ARM 实机编译器流水线截图

在 ARM64 设备上执行自动化测试并重新生成 qsort 各阶段输出：

```
python3 -m unittest discover -s tests -p "test_*.py"
python3 -m qcompiler samples/qsort.q1 -o outputs/qsort
wc -l outputs/qsort/qsort.tokens.txt outputs/qsort/qsort.ast.json outputs/qsort/qsort.qtac outputs/qsort/qsort.s
ls -lh outputs/qsort
```

```
(base) root@orangeipiaipro:~/Codes# cd lab2/
(base) root@orangeipiaipro:~/Codes/lab2# python3 -m unittest discover -s tests -p "test_*.py"
.....

Ran 7 tests in 0.104s

因为当前实现按 4 字节整数处理，所以 1395 字节数据区分配。最后它也是为了证明：全局数组声明被正确译成 ARM64 汇编中的数据区分配。
OK
(base) root@orangeipiaipro:~/Codes/lab2# python3 -m qcompiler samples/qsrt.q -o outputs/qsrt
generated outputs/qsrt
(base) root@orangeipiaipro:~/Codes/lab2# wc -l outputs/qsrt/qsrt.tokens.txt outputs/qsrt/qsrt.ast.json outputs/qsrt/qsrt.qtac outputs/qsrt/qsrt.s
235 outputs/qsrt/qsrt.tokens.txt
568 outputs/qsrt/qsrt.ast.json
89 outputs/qsrt/qsrt.qtac
274 outputs/qsrt/qsrt.s
229 outputs/qsrt/qsrt_opt.s
1395 total
(base) root@orangeipiaipro:~/Codes/lab2# ls -lh outputs/qsrt
total 44K
-rw-r--r-- 1 root root 15K Jun 12 01:34 qsrt.ast.json
-rw-r--r-- 1 root root 1.1K Jun 12 01:34 qsrt.qtac
-rw-r--r-- 1 root root 5.1K Jun 12 01:34 qsrt.s
-rw-r--r-- 1 root root 195 Jun 12 01:34 qsrt.symbols.txt
-rw-r--r-- 1 root root 3.5K Jun 12 01:34 qsrt.tokens.txt
-rw-r--r-- 1 root root 4.3K Jun 12 01:34 qsrt_opt.s
(base) root@orangeipiaipro:~/Codes/lab2#
```

11.7 未优化 ARM 汇编、链接与运行截图

对未优化汇编 `qsrt.s` 做汇编、链接和运行：

```
as -o outputs/qsrt/qsrt.o outputs/qsrt/qsrt.s
ld -o outputs/qsrt/qsrt outputs/qsrt/qsrt.o
file outputs/qsrt/qsrt
./outputs/qsrt/qsrt
echo $?
```

```
(base) root@orangeipiaipro:~/Codes/lab2# as -o outputs/qsrt/qsrt.o outputs/qsrt/qsrt.s
(base) root@orangeipiaipro:~/Codes/lab2# ld -o outputs/qsrt/qsrt outputs/qsrt/qsrt.o
(base) root@orangeipiaipro:~/Codes/lab2# file outputs/qsrt/qsrt
outputs/qsrt/qsrt: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), statically linked, not stripped
(base) root@orangeipiaipro:~/Codes/lab2# ./outputs/qsrt/qsrt
(base) root@orangeipiaipro:~/Codes/lab2# echo $?
0
(base) root@orangeipiaipro:~/Codes/lab2#
```

可以看到结果符合预期：

1. `as` 和 `ld` 没有报错。
2. `file outputs/qsrt/qsrt` 显示该文件为 ARM64 或 AArch64 可执行文件。
3. 程序运行后 `echo $?` 输出 `0`，说明正常退出。

11.8 优化后 ARM 汇编、链接与运行截图

对优化后汇编 `qsrt_opt.s` 做同样验证：

```
as -o outputs/qsrt/qsrt_opt.o outputs/qsrt/qsrt_opt.s
ld -o outputs/qsrt/qsrt_opt outputs/qsrt/qsrt_opt.o
file outputs/qsrt/qsrt_opt
./outputs/qsrt/qsrt_opt
echo $?
```

```

(base) root@orangeipiaipro:~/Codes/lab2# as -o outputs/qsort/qsort_opt.o outputs/qsort/qsort_opt.s
(base) root@orangeipiaipro:~/Codes/lab2# ld -o outputs/qsort/qsort_opt outputs/qsort/qsort_opt.o
file outputs/qsort/qsort_opt
./outputs/qsort/qsort_opt
echo $?
outputs/qsort/qsort_opt: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), statically linked, not stripped
0
(base) root@orangeipiaipro:~/Codes/lab2#

```

得到相似结果。

11.9 程序正确性验证

```

(base) root@orangeipiaipro:~/Codes/lab2# gdb -q outputs/qsort/qsort
Reading symbols from outputs/qsort/qsort...
(No debugging symbols found in outputs/qsort/qsort)
(gdb) set pagination off
(gdb) b _start
Breakpoint 1 at 0x400468
(gdb) run
Starting program: /root/Codes/lab2/outputs/qsort/qsort

Breakpoint 1, 0x000000000400468 in _start ()
(gdb) ni
0x00000000040046c in _start ()
(gdb) set $base = (char *)$x0
(gdb) set {long long}($base+0) = 9
(gdb) set {long long}($base+8) = 1
(gdb) set {long long}($base+16) = 8
(gdb) set {long long}($base+24) = 2
(gdb) set {long long}($base+32) = 7
(gdb) set {long long}($base+40) = 3
(gdb) set {long long}($base+48) = 6
(gdb) set {long long}($base+56) = 4
(gdb) set {long long}($base+64) = 5
(gdb) set {long long}($base+72) = 0
(gdb) x/10gd $base
0x410488:    9      1
0x410498:    8      2
0x4104a8:    7      3
0x4104b8:    6      4
0x4104c8:    5      0
(gdb) ni 3
0x000000000400478 in _start ()
(gdb) x/10gd $base
0x410488:    0      1
0x410498:    2      3
0x4104a8:    4      5
0x4104b8:    6      7
0x4104c8:    8      9
(gdb) quit
A debugging session is active.

```

可以看到，用gdb调试显示，原版的 `qsort` 能够正常给数组排序，最后变成了 `0~9` 的顺序。

```
(base) root@orangepiaipro:~/Codes/lab2# gdb -q outputs/qsort/qsort_opt
x/10gd $base
ni 3
x/10gd $base
"on" or "off" expected.
(gdb) set pagination off
(gdb) b _start
Breakpoint 1 at 0x4003b4
(gdb) run
Starting program: /root/Codes/lab2/outputs/qsort/qsort_opt

Breakpoint 1, 0x0000000004003b4 in _start ()
(gdb) ni
0x0000000004003b8 in _start ()
(gdb) set $base = (char *)$x0
(gdb) set {long long}($base+0) = 9
(gdb) set {long long}($base+8) = 1
(gdb) set {long long}($base+16) = 8
(gdb) set {long long}($base+24) = 2
(gdb) set {long long}($base+32) = 7
(gdb) set {long long}($base+40) = 3
(gdb) set {long long}($base+48) = 6
(gdb) set {long long}($base+56) = 4
(gdb) set {long long}($base+64) = 5
(gdb) set {long long}($base+72) = 0
(gdb) x/10gd $base
0x4103d8:      9      1
0x4103e8:      8      2
0x4103f8:      7      3
0x410408:      6      4
0x410418:      5      0
(gdb) ni 3
0x0000000004003c4 in _start ()
(gdb) x/10gd $base
0x4103d8:      0      1
0x4103e8:      2      3
0x4103f8:      4      5
0x410408:      6      7
0x410418:      8      9
(gdb)
```

`qsort_opt` 也一切正常。排序效果一致。

十二、AI 使用方式及效果

本实验使用 AI 辅助完成以下工作：

1. 阅读实验 PDF，进一步理解实验要求。
2. 阅读并复用 Lab1 中的 QTAC 到 ARM64 代码生成思路。
3. 编写自动化测试，发现并修复函数入口标签重复、函数内局部标签冲突等问题。
4. 整理实验报告。

AI 对本实验的主要帮助体现在工程组织和调试效率上。通过自动化测试，能够快速定位前端、QTAC 生成和后端代码生成之间的接口问题。

十三、问题解决与遗留问题

13.1 已解决问题

1. `qsort` 函数调用使用尾随逗号形式，例如 `qsort(a[], 0, 9,);`，解析器已支持该形式。
2. QTAC 中函数体包含 `LABEL function;`，ARM 后端同时会输出函数入口标签，最初导致重复标签。后续在 ARM 后端跳过与当前函数名相同的入口标签。
3. 不同函数中都会出现 `l0`、`l1` 等局部标签，最初可能导致汇编符号冲突。后续改为输出 `.L_<function>_<label>` 形式。
4. 初始优化 pass 只能删除少量跳转，效果不明显。后续增加了乘以 8 到左移 3 位的强度削弱，以及相邻 `scratch` 寄存器指令合并，使优化后汇编从 274 行减少到 229 行。

5. qsort 源程序本身没有标准输入输出。为验证排序语义，本实验在 ARM 实机上使用 GDB 向全局数组 `a` 写入测试数据，并在 `qsort` 返回后检查数组内存，验证未优化和优化后程序均能得到 `0~9` 的有序结果。

13.2 遗留问题

1. 当前 QL 子集覆盖 qsort 和实验核心结构，但没有扩展浮点数、字符串、结构体或超过 8 个参数的函数调用。
2. 当前优化 pass 仍属于局部窥孔优化，即只针对相邻几条 ARM64 指令做模式替换，例如将乘以 8 的地址计算替换为左移 3 位，还没有实现公共子表达式消除、复制传播、活跃变量分析或图着色寄存器分配。
3. 当前 QL 程序没有 `printf`、`fprintf`、标准输入输出或运行时库支持，排序结果通过 GDB 内存检查验证，而不是由目标程序直接打印。

十四、实验收获

本实验将编译原理中的多个阶段串联为一个可运行工程。相比只实现单独的词法或语法分析，本实验更能体现不同阶段之间接口设计的重要性。

主要收获如下：

1. 词法分析阶段不仅要识别 Token，还要处理注释、特殊字符和位置信息。
2. 递归下降分析器虽然实现直接，但需要清楚设计表达式优先级和语句入口判断。
3. QAST 是前端和中间代码生成之间的重要边界，清晰的节点结构可以降低后续翻译难度。
4. 数组访问在中间代码层需要显式展开为地址计算和内存访问。
5. 后端生成汇编时，函数标签、局部标签、调用约定和栈帧布局都必须严格处理，否则容易出现链接或运行错误。
6. 自动化测试能够及时发现代码生成中的符号冲突问题，比只人工观察输出更可靠。
7. 汇编优化需要兼顾效果和语义安全。强度削弱可以明显减少数组地址计算指令，但优化规则必须限制在安全范围内，避免破坏函数参数保存等必要指令。
8. 目标程序没有输出时，仍然可以通过 GDB 向全局数组写入测试数据并检查运行后的内存，验证生成程序的排序语义。

十五、结论

本次实验完成了 QCompiler 的主要编译流程，实现了从 QL 源程序到 Token、QAST、QTAC、符号表、ARM64 汇编和优化后 ARM64 汇编的生成。qsort 测试程序已经通过自动化测试，并在 ARM 实机上完成未优化和优化后汇编的汇编、链接、运行验证。

针对实验要求中的“验证排序功能正确性”，本实验进一步使用 GDB 在程序运行前向全局数组 `a` 写入乱序数据，并在 `qsort` 返回后检查数组内存。截图结果显示未优化程序和优化后程序均能将数组排序为 `0~9`，说明目标代码不仅能够正常执行，也保留了 qsort 的排序语义。

整体来看，本实现覆盖了任务 A 的核心编译阶段，并完成了简单语义保持汇编优化。后续如果继续完善，可以扩展 QL 的标准输入输出能力、实现更系统的优化 pass，并进一步增强类型检查和运行时支持。